Python: exercises on files

This notebook was generated in **solutions** mode.

Exercise 1: Writing and Reading a File

Exercise Description

Write a program that:

1. Creates and opens a file named `"students.txt"` in write mode.
2. Writes the names of five students, each on a new line, to the file.
3. Closes the file.
4. Reopens the file in read mode and prints each student's name.

Store the result in a variable `result` containing a list of the student names read from the file.

Your solution

```
# Step 1: Create and open the file in write mode
with open("students.txt", "w") as f:
    # Write each student name on a new line
```

```
    f.write("Alice\n")
    f.write("Bob\n")
    f.write("Charlie\n")
    f.write("David\n")
    f.write("Eve\n")

# Step 2: Open the file in read mode and print each student's name
result = []
with open("students.txt", "r") as f:
    for line in f:
        name = line.strip()  # strip() removes any extra newline characters
        result.append(name)
        print(name)
```

Test your solution

```
# Test that result contains the expected student names
assert len(result) == 5
assert "Alice" in result
assert "Bob" in result
assert "Charlie" in result
assert "David" in result
assert "Eve" in result
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Storing and Loading Scores

Exercise Description

1. Create a list of five scores: `[85, 90, 78, 92, 88]`.
2. Write a program that:
    - Opens a file named `"scores.txt"` in write mode.
    - Writes each score from the list to the file, with each score on a new line.
    - Closes the file.
3. Reopen the file in read mode and:
    - Read all lines from the file, convert each line to an integer, and store the results in a new list.
    - Store the loaded scores in a variable `loaded_scores`.

Your solution

```
# Step 1: Create a list of scores
scores = [85, 90, 78, 92, 88]

# Step 2: Write each score to the file, each on a new line
with open("scores.txt", "w") as f:
    for score in scores:
        f.write(f"{score}\n")

# Step 3: Read scores back from the file and store them in a list
loaded_scores = []
with open("scores.txt", "r") as f:
    for line in f:
        loaded_scores.append(int(line.strip()))  # Convert each line to an integer
```

```
# Print the list of scores loaded from the file
print("Loaded scores:", loaded_scores)
```

Test your solution

```
# Test that loaded_scores matches the original scores
assert loaded_scores == [85, 90, 78, 92, 88]
assert len(loaded_scores) == 5
assert all(isinstance(score, int) for score in loaded_scores)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Appending to a File

Exercise Description

Write a program that:

1. Opens a file named `"notes.txt"` in append mode.
2. Adds three predefined notes to the file: "First note", "Second note", "Third note".
3. Each note should be written on a new line.
4. Finally, opens the file in read mode and stores all notes in a variable `all_notes` as a list.

Store the result in a variable `all_notes` containing a list of all the notes read from the file.

Your solution

```
# Step 1: Define the notes to be added
notes = ["First note", "Second note", "Third note"]

# Step 2: Open the file in append mode to add new notes
with open("notes.txt", "a") as f:
    # Add each note to the file
    for note in notes:
        f.write(note + "\n")

# Step 3: Open the file in read mode and print all notes
all_notes = []
with open("notes.txt", "r") as f:
    print("All notes:")
    for line in f:
        note = line.strip()
        all_notes.append(note)
        print(note)
```

Test your solution

```
# Test that all_notes contains the expected notes
assert "First note" in all_notes
```

```
assert "Second note" in all_notes
assert "Third note" in all_notes
assert len(all_notes) >= 3  # May contain more if file existed before
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Counting Words in a File

Exercise Description

Write a program that:

1. Uses the predefined sentence: "Python is a powerful programming language".
2. Opens a file named `"sentence.txt"` in write mode and writes the sentence to the file.
3. Closes the file.
4. Reopens the file in read mode, reads the sentence back, and counts the number of words.
5. Stores the word count in a variable `word_count`.

Hint: Use `split()` on the sentence to count the words.

Your solution

```
# Step 1: Define the sentence to be written to the file
sentence = "Python is a powerful programming language"
with open("sentence.txt", "w") as f:
    f.write(sentence)

# Step 2: Read the sentence back from the file and count the words
with open("sentence.txt", "r") as f:
    sentence_from_file = f.read()  # Read the whole sentence

# Count the words by splitting the sentence into a list of words
word_count = len(sentence_from_file.split())
print("Word count:", word_count, sentence_from_file.split())
```

Test your solution

```
# Test that word_count is correct
assert word_count == 6  # "Python is a powerful programming language" has 6 words
assert isinstance(word_count, int)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Saving and Loading a Shopping List

Exercise Description

Write a program that:

1. Uses a predefined shopping list: `["apple", "bread", "milk"]`.
2. Opens a file named `"shopping_list.txt"` in write mode and writes each item from the list to a new line.
3. Reopens the file in read mode and:
    - Reads all lines into a list.
    - Stores the shopping list from the file in a variable `loaded_shopping_list`.

Your solution

```
# Step 1: Define the shopping list
shopping_list = ["apple", "bread", "milk"]

# Step 2: Write each item to the file, each on a new line
with open("shopping_list.txt", "w") as f:
    for item in shopping_list:
        f.write(item + "\n")

# Step 3: Read the shopping list back from the file and print it
loaded_shopping_list = []
print("Shopping List from File:")
with open("shopping_list.txt", "r") as f:
    for line in f:
        item = line.strip()
```

```
        loaded_shopping_list.append(item)
        print(item)
```

Test your solution

```
# Test that loaded_shopping_list matches the original
assert loaded_shopping_list == ["apple", "bread", "milk"]
assert len(loaded_shopping_list) == 3
assert "apple" in loaded_shopping_list
assert "bread" in loaded_shopping_list
assert "milk" in loaded_shopping_list
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Saving Calculated Results to a File

Exercise Description

1. Create a list of numbers: `[5, 10, 15, 20]`.
2. Write a program that:
    - Calculates the square of each number and writes the results to a file named `"squares.txt"`.
    - Each square should be written on a new line.
3. Open the file in read mode and store each square in a variable `squares_list` as a list of integers.

Your solution

```
# Step 1: Define the list of numbers
numbers = [5, 10, 15, 20]

# Step 2: Calculate squares and write to file
with open("squares.txt", "w") as f:
    for number in numbers:
        square = number ** 2
        f.write(f"{square}\n")

# Step 3: Read and print each square from the file
squares_list = []
with open("squares.txt", "r") as f:
    print("Squares:")
    for line in f:
        square = int(line.strip())
        squares_list.append(square)
        print(line.strip())
```

Test your solution

```python
# Test that squares_list contains the correct squares
assert squares_list == [25, 100, 225, 400]  # 5^2, 10^2, 15^2, 20^2
assert len(squares_list) == 4
assert all(isinstance(square, int) for square in squares_list)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Reversing Lines in a File

Exercise Description

Write a program that:

1. Uses five predefined lines of text: `["Line 1", "Line 2", "Line 3", "Line 4", "Line 5"]`.
2. Saves each line to a file named `"reversed_lines.txt"`.
3. Opens the file in read mode and prints each line in reverse order.
4. Stores the reversed lines in a variable `reversed_lines` as a list.

Your solution

```
# Step 1: Define the lines to be written
lines = ["Line 1", "Line 2", "Line 3", "Line 4", "Line 5"]

# Step 2: Get user input and write each line to the file
with open("reversed_lines.txt", "w") as f:
    for line in lines:
        f.write(line + "\n")

# Step 3: Read each line and print in reverse order
reversed_lines = []
with open("reversed_lines.txt", "r") as f:
    lines_from_file = f.readlines()
    print("Reversed Lines:")
    for line in reversed(lines_from_file):
        stripped_line = line.strip()
        reversed_lines.append(stripped_line)
        print(stripped_line)
```

Test your solution

```
# Test that reversed_lines contains lines in reverse order
assert reversed_lines == ["Line 5", "Line 4", "Line 3", "Line 2", "Line 1"]
assert len(reversed_lines) == 5
assert reversed_lines[0] == "Line 5"
assert reversed_lines[-1] == "Line 1"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: Finding the Longest Word in a File

Exercise Description

Write a program that:

1. Uses the predefined sentence: "Programming with Python is incredibly rewarding".
2. Writes the sentence to a file named `"sentence.txt"`.
3. Opens the file in read mode, reads the sentence back, and finds the longest word.
4. Stores the longest word in a variable `longest_word`.

Your solution

```
# Step 1: Define the sentence and write it to the file
sentence = "Programming with Python is incredibly rewarding"
with open("sentence.txt", "w") as f:
    f.write(sentence)

# Step 2: Read the sentence and find the longest word
with open("sentence.txt", "r") as f:
    words = f.read().split()
    longest_word = max(words, key=len)

print("Longest word:", longest_word)
```

Test your solution

```
# Test that longest_word is correct
assert longest_word == "Programming"  # "Programming" is the longest word (11 charact
assert len(longest_word) == 11
assert isinstance(longest_word, str)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Reading Data and Calculating the Average

Exercise Description

1. Create a file named `"data.txt"` containing numerical values: `[10, 15, 20, 25, 30]`, each on a new line.
2. Write a program that:
    - Reads each value from the file.
    - Calculates the average of the values.
3. Store the calculated average in a variable `average`.

Your solution

```python
# Step 1: Create the file with sample data
data_values = [10, 15, 20, 25, 30]
with open("data.txt", "w") as f:
    for value in data_values:
        f.write(f"{value}\n")

# Step 2: Read each value and calculate the average
values = []
with open("data.txt", "r") as f:
    for line in f:
        values.append(int(line.strip()))

average = sum(values) / len(values)
print("Average value:", average)
```

Test your solution

```python
# Test that average is calculated correctly
assert average == 20.0  # (10+15+20+25+30)/5 = 100/5 = 20.0
assert isinstance(average, float)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Saving a List of Names in Uppercase

Exercise Description

Write a program that:

1. Uses five predefined names: `["alice", "bob", "charlie", "diana", "eve"]`.
2. Saves each name in uppercase to a file named `"uppercase_names.txt"`, with each name on a new line.
3. Opens the file in read mode and stores each uppercase name in a variable `uppercase_names` as a list.

Your solution

```
# Step 1: Define the names and save in uppercase
names = ["alice", "bob", "charlie", "diana", "eve"]
with open("uppercase_names.txt", "w") as f:
    for name in names:
        uppercase_name = name.upper()
        f.write(uppercase_name + "\n")

# Step 2: Read and print each uppercase name
uppercase_names = []
with open("uppercase_names.txt", "r") as f:
    print("Names in uppercase:")
    for line in f:
        name = line.strip()
        uppercase_names.append(name)
        print(name)
```

Test your solution

```
# Test that uppercase_names contains the correct uppercase names
assert uppercase_names == ["ALICE", "BOB", "CHARLIE", "DIANA", "EVE"]
assert len(uppercase_names) == 5
assert all(name.isupper() for name in uppercase_names)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 11: Writing and Checking for Palindromes

Exercise Description

Write a program that:

1. Uses five predefined words: `["racecar", "hello", "level", "python", "radar"]`.
2. Saves each word to a file named `"words.txt"`.
3. Reopens the file, reads each word, and stores only those words that are palindromes (words that read the same forwards and backwards) in a variable `palindromes` as a list.

Your solution

```
# Step 1: Define the words and save to file
words = ["racecar", "hello", "level", "python", "radar"]
with open("words.txt", "w") as f:
    for word in words:
        f.write(word + "\n")

# Step 2: Read each word and print if it's a palindrome
palindromes = []
print("Palindromes:")
with open("words.txt", "r") as f:
    for line in f:
        word = line.strip()
        if word == word[::-1]:  # Check if word reads the same forwards and backwards
            palindromes.append(word)
            print(word)
```

Test your solution

```
# Test that palindromes contains only the palindromic words
assert set(palindromes) == {"racecar", "level", "radar"}  # Only palindromes
assert len(palindromes) == 3
assert "hello" not in palindromes  # Not a palindrome
assert "python" not in palindromes  # Not a palindrome
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 12: Counting Specific Words in a Text File

Exercise Description

1. Create a file named `"story.txt"` containing a short paragraph with predefined text.
2. Write a program that:
    - Uses the word "the" as the target word to count.
    - Reads the contents of the file and counts how many times the word appears in the text (case-insensitive).
3. Store the total count in a variable `word_count`.

Your solution

```
# Step 1: Write a sample paragraph to the file
story_text = ["Once upon a time in a land far, far away there was a brave knight.",
              "The knight set out on an adventure to rescue the kingdom."]
with open("story.txt", "w") as f:
    for sentence in story_text:
        f.write(sentence + "\n")

# Step 2: Define the word to search for
word_to_count = "the"

# Step 3: Count the occurrences of the word in the file
word_count = 0
with open("story.txt", "r") as f:
```

```
    for line in f:
        words = line.lower().split()
        word_count += words.count(word_to_count.lower())

print(f"The word '{word_to_count}' appears {word_count} times.")
```

Test your solution

```
# Test that word_count is correct
assert word_count == 2  # "the" appears twice in the story ("The knight" and "the kir
assert isinstance(word_count, int)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 13: Writing a Simple CSV File

Exercise Description

Write a program that:

1. Uses predefined information for three people: `[("Alice", "25", "alice@email.com"), ("Bob", "30", "bob@email.com"), ("Charlie", "35", "charlie@email.com")]`.
2. Writes this information to a file named `"people.csv"`, with each field separated by commas and each person on a new line.
3. Includes a header row: "Name,Age,Email".
4. Opens the file in read mode and stores each line in a variable `csv_lines` as a list.

Your solution

```
# Step 1: Define the people data
people_data = [("Alice", "25", "alice@email.com"), ("Bob", "30", "bob@email.com"), ('

# Step 2: Write people details to CSV
with open("people.csv", "w") as f:
    f.write("Name,Age,Email\n")  # Write header row
    for name, age, email in people_data:
        f.write(f"{name},{age},{email}\n")

# Step 3: Read and print each line in the CSV to verify format
csv_lines = []
with open("people.csv", "r") as f:
    print("CSV Content:")
    for line in f:
        line_content = line.strip()
        csv_lines.append(line_content)
        print(line_content)
```

Test your solution